

LYTA — Security Audit CONFIDENTIEL

LYTA — Security Audit (CONFIDENTIEL)

Version 1.0 — 8 juin 2026 **Classification**  **CONFIDENTIEL** — **NDA renforcé requis** **Audience** Développeur externe missionné pour audit sécurité LYTA + Habib Agharbi **Document associé (public)**

LYTA_Developer_Onboarding.md

AVERTISSEMENT CRITIQUE

Ce document décrit : - Les **surfaces d'attaque** de la plateforme LYTA - Les **endpoints publics** et leurs mécanismes d'auth - Les **secrets** et leur emplacement - L'**état réel** de l'isolation multi-tenant - Les **vulnérabilités connues ou suspectes** non encore corrigées

Si ce document fuit, il constitue un mode d'emploi pour attaquer la plateforme.

Règles d'usage

-  Lecture sur poste physique sécurisé
-  Discussion entre le dev signataire NDA renforcé et Habib uniquement
-  Pas de partage par email non chiffré
-  Pas de copy-paste dans un outil tiers (Notion, Slack, Discord, IDE cloud, ChatGPT...)
-  Pas de versioning dans un repo public ou un fork
-  Destruction obligatoire à la fin de la mission

Périmètre temporel

Ce document reflète l'état du code au **8 juin 2026 (commit `0587c16`)**. À chaque évolution majeure de sécurité (nouvelle edge function publique, refactor RLS, rotation de secrets), il doit être mis à jour ou réémis.

Table des matières

1. Surface d'attaque inventaire
2. Auth model (détail)

- 3. RLS deep dive
- 4. Edge functions sans JWT — audit
- 5. Secrets management
- 6. Multi-tenant isolation analysis
- 7. Vulnérabilités connues ou suspectes
- 8. Recommandations prioritaires

1. Surface d’attaque inventaire

1.1 URLs publiques (accessibles sans auth)

URL	But	Risque
app.lyta.ch	Landing + connexion	Login bruteforce (limité par rate-limit Supabase Auth)
*.lyta.ch	Connexion par tenant	Idem
*.lyta.ch/signer/:token	Page signature publique	Si token leak → signature usurpée
*.lyta.ch/deposer-contrat	Dépôt contrat public	Spam / injection si pas de rate-limit
*.lyta.ch/reset-password	Reset password	Magic link leak
lyta.ch/access (Lovable séparé)	Self-signup post-Stripe	Session_id Stripe leak
Edge functions Supabase	Voir §4	Variable selon fonction

1.2 Edge functions publiques (verify_jwt = false)

Toutes les 59 fonctions sont déclarées `verify_jwt = false` dans `config.toml`. Cela ne veut pas dire qu’elles sont toutes publiques — chacune doit gérer son auth en interne. Voir §4 pour l’audit individuel.

1.3 Buckets Storage

Bucket	Type	Policies
documents	Privé	Tenant isolation via RLS storage (<code>storage.objects</code>)
public-deposits	Public ?	À auditer — uploads publics pour <code>deposit-contract</code>
Logos compagnies	Public read	Acceptable (assets statiques)
Logos tenants	Public read	Acceptable (assets affichés sur landing tenant)

1.4 Variables d'environnement publiques (frontend)

⚠️ `VITE_SUPABASE_ANON_KEY` est exposée dans le bundle JS servi au navigateur. C'est attendu (clé anon publique de Supabase), mais ça signifie : - N'importe qui peut faire un client supabase-js avec cette clé. - **Toute la sécurité repose sur les RLS** + les checks d'auth dans les edge functions. - Un dev malveillant ayant cette clé peut tester systématiquement chaque table/RPC pour trouver une porte ouverte.

1.5 Domaine custom tenant (`<slug>.lyta.ch`)

- DNS Cloudflare → Vercel via CNAME `cname.vercel-dns.com`
- Provisioning automatique via `tenant-onboarding`
- **Risque** : si un attaquant peut faire passer un domaine à `tenant-onboarding` avec un slug usurpé → potentielle prise de contrôle d'un sous-domaine. **À auditer** : qui peut appeler `tenant-onboarding` et comment elle valide le slug.

2. Auth model (détail)

2.1 Flow auth standard (collaborateur)

1. User → Connexion : POST email + password
2. `supabase.auth.signInWithPassword({ email, password })`
 - Supabase Auth (GoTrue) vérifie le hash bcrypt
 - Retourne { session: { access_token JWT, refresh_token, expires_at } }
3. supabase-js stocke la session en localStorage
4. Toutes les requêtes ajoutent automatiquement Authorization: Bearer <jwt>
5. PostgREST decode le JWT (vérif signature HS256 avec SUPABASE_JWT_SECRET)
 - Extrait `auth.uid()` (sub claim)
 - Applique RLS basé sur `auth.uid()`

2.2 Flow MFA SMS (custom LYTA)

⚠️ **Implémentation custom** au-dessus de Supabase Auth (la MFA SMS native Supabase n'est pas utilisée).

1. `signInWithPassword` réussit → user a une session VALIDE Supabase
2. LYTA détecte ``tenant_security_settings.require_sms_mfa = true``
3. Appelle `send-verification-sms` avec le téléphone du profil
4. State ``pendingSmsVerification`` stocké en `sessionStorage`
5. User saisit code → `verify-sms-code` valide via Twilio Verify API
6. Si OK → la session du `smsClient` est migrée vers le client principal

🔴 Faille potentielle (CRITIQUE)

Entre les étapes 1 et 6, la session Supabase est techniquement valide. Si l'attaquant intercepte le `access_token` à ce stade (XSS, extension malveillante, malware local), il peut bypasser la vérification SMS en utilisant le token directement.

Mitigation actuelle : la session est mise dans un `smsClient` avec storage isolée (sessionStorage), pas local-Storage. Mais le JWT reste valide côté Supabase.

Recommandation : sur les tenants avec MFA SMS obligatoire, **shorter le JWT TTL à 5-10 minutes** (`jwt_expiry`) pour limiter la fenêtre.

2.3 Flow auth client final (portail)

1. Client → reçoit magic link par email (créé par send-client-message ou autre)
2. Click → Supabase Auth valide le magic link → crée session
3. ClientLayout.tsx:88 vérifie `SELECT FROM clients WHERE user_id = auth.uid()`
 - Si trouve une ligne → accès portal
 - Sinon → redirection

Pas de password requis pour les clients finaux — ils n'utilisent que magic link (à confirmer). Si vrai → pas de bruteforce password possible mais magic link = email = phishing surface.

2.4 Flow auth signataire (`/signer/:token`)

1. Client reçoit email avec lien `/signer/<UUID_token>`
2. Page Signer charge sans auth
3. Appelle RPC `get_signature_request_by_token(token)`
 - Si trouvé + non expiré + status valide → affiche le doc
4. Signature → `complete-signature(token, signed_pdf_base64)`

Auth = token UUID + state (expires_at, status). Pas de lien avec `auth.users`. Le client peut signer sans avoir de compte Supabase.

3. RLS deep dive

3.1 Fonctions helper SECURITY DEFINER

LYTA s'appuie massivement sur des **fonctions SECURITY DEFINER** pour ses RLS. Avantages : - Évite la récursion infinie (un check qui fait SELECT sur la même table avec RLS) - Permet de hoister le calcul (Postgres planner peut cacher la valeur) - Réduit la duplication dans les policies

Fonction	Type	Cache	Usage
<code>get_user_tenant_id()</code>	STABLE	session	retourne tenant_id actif
<code>my_collab_id_for_active_tenant()</code>	STABLE	session	id collaborateur
<code>my_collab_id_v2()</code>	STABLE PARALLEL SAFE	session	optim version
<code>has_role(text)</code>	STABLE	session	rôle global
<code>has_global_scope_v2()</code>	STABLE PARALLEL SAFE	session	scope global
<code>is_crm_member_of_tenant(uuid)</code>	STABLE	session	appartenance
<code>can_access_client(uuid)</code>	STABLE	per-row	accès client (composite)
<code>_client_is_in_team(uuid, uuid)</code>	STABLE	per-row	scope team
<code>can_view_financial_data()</code>	STABLE	session	droit voir commissions
<code>can_see_commissions_scope()</code>	STABLE	session	scope commissions
<code>apply_retrocommission(...)</code>	VOLATILE	—	calcul retro (RPC)
<code>calculate_commission_with_rules(...)</code>	VOLATILE	—	calcul commission (RPC)

3.2 Politiques clés par table critique

clients

```

-- SELECT
CREATE POLICY "clients_select" ON public.clients
FOR SELECT
USING (tenant_id = get_user_tenant_id());
-- Le scope team/personal est appliqué côté frontend dans useClients.tsx
-- (cause : RLS scope-aware = timeout 80s en SQL – révert juin 2026)

-- INSERT
CREATE POLICY "clients_insert" ON public.clients
FOR INSERT
WITH CHECK (tenant_id = get_user_tenant_id() AND is_crm_member_of_tenant(tenant_id));
-- ⚠ Cette policy plante en 42501 en runtime → bypass via create-client edge function

```

signature_requests

```
-- SELECT pour membres tenant
CREATE POLICY "sig_req_select_tenant" ON public.signature_requests
FOR SELECT
USING (tenant_id = get_user_tenant_id() AND is_crm_member_of_tenant(tenant_id));

-- SELECT par token (signer page, sans auth)
-- → Pas de policy directe : la fonction RPC get_signature_request_by_token
-- est SECURITY DEFINER et bypass RLS, mais vérifie le token + status + expiration
```

documents

```
-- SELECT : via can_access_client(client_id) qui compose tenant + scope
CREATE POLICY "documents_select" ON public.documents
FOR SELECT
USING (can_access_client(client_id) OR is_crm_member_of_tenant(tenant_id));
```

commissions et commission_*

⚠ Les politiques dépendent du droit financier — `can_view_financial_data()` qui combine rôle + permissions granulaires. **À auditer** : un agent peut-il voir SES commissions mais pas celles des autres ? Si oui via quelle clause ?

3.3 Storage RLS

`storage.objects` a 4 policies pour le bucket `documents` : - SELECT/INSERT/UPDATE/DELETE → “Tenant users can ...”

Le filtrage se fait via le **path** du fichier : convention `<broker_user_id>/<sous-dossier>/<file>`. Les policies extraient le `<broker_user_id>` du path et vérifient qu’il appartient au tenant actif. **À confirmer en lisant les policies** (migration `20260315000001_storage_tenant_isolation.sql`).

Risque

Si un user peut **lister** les objets en passant par un path qui n’est pas le sien, fuite cross-tenant possible. À tester :

```
# Test : avec un user du tenant A, essayer de download
# un fichier du tenant B en connaissant son path
curl -H "Authorization: Bearer <jwt_A>" \
  "https://...supabase.co/storage/v1/object/documents/<broker_B>/private/file.pdf"
```

4. Edge functions sans JWT — audit

Toutes les fonctions sont `verify_jwt = false`. Voici l’audit fonctionnel :

4.1 Fonctions REQUIREAUTH (auth en interne)

Ces fonctions appellent `requireAuth(req)` qui valide le JWT manuellement (via `supabase.auth.getUser(token)`). **C'est OK** tant que le check est appelé en premier.

✅ `create-client`, `bypass-insert`, `save-policy`, `send-crm-email`, `send-client-message`, `create-collaborator`, `add-user-seat`, `cancel-tenant-subscription`, `dispatch-mandat-to-companies`, `send-signature-invite`, `scan-document`, `classify-batch-documents`, `scan-commission-statement`, `send-lpp-search-requests`, `ai-chat`, `submit-referral`

Risque : si une de ces fonctions oublie le `requireAuth()` au début OU le fait après une action sensible → bypass. **À auditer ligne par ligne** dans une seconde phase.

4.2 Fonctions KING-only

Vérification que le caller est `king`. **À confirmer** que c'est bien implémenté de manière homogène (pas un check ad-hoc différent dans chaque fonction).

🟡 `delete-tenant`, `delete-user-account`, `activate-tenant`, `reset-tenant-data`, `export-tenant-data`, `king-impersonate-tenant`, `king-stripe-stats`, `list-tenant-invoices`, `sync-tenant-stripe`, `sync-external-billing`, `resend-signup-finalization`, `send-test-tenant-emails`

🔴 **Faible potentielle** : `king-impersonate-tenant`

Cette fonction génère un magic link de connexion comme admin d'un tenant. Si : - Le check KING est par signature JWT seul → un attaquant qui a un JWT KING (vol, vieille session) peut impersonate n'importe quel tenant - Le check n'est pas time-limité (durée du magic link généré ?) - Pas de notification visible côté tenant impersoné

Recommandation : ajouter un check 2FA fresh (TOTP < 5 min) pour cette fonction critique.

4.3 Fonctions publiques avec token UUID

Sécurité = token UUID non-devinable + state DB (expiration, status).

🟡 `proxy-signature-pdf`, `get-signature-pdf-url`, `complete-signature`, `request-signature-link-renewal`

Risque : si le token leak (logs, email cache, share by mistake) → accès au document.

Mitigation : expiration 24h, `status` (cancelled = bloqué), audit `mark_signature_request_viewed`.

4.4 Fonctions publiques avec session Stripe

🟡 `provision-self-signup-tenant`, `get-checkout-session-info`, `create-checkout-session`

Sécurité = `session_id` Stripe. Le `session_id` est secret et retourné par Stripe à la fin du paiement.

Risque : il apparaît dans l'URL du browser de l'user, donc dans son historique + dans les logs serveur.

Recommandation : marquer une session comme "consommée" après le premier `provision-self-signup-tenant` qui réussit → empêche un replay si quelqu'un récupère l'URL.

4.5 Fonctions publiques rate-limited

● `check-slug-availability`, `swiss-postal-code-lookup`, `swiss-address-lookup`, `health-check`

Endpoints informatifs sans data sensible. Rate-limit Supabase suffit. **OK**.

4.6 Crons / service_role only

● `send-birthday-emails`, `send-renewal-reminders`, `send-follow-up-reminders`, `process-scheduled-emails`, `apply-monthly-overage`

Appelés via `pg_net` ou cron Supabase. Le check `service_role` est implicite (impossible d'appeler depuis un client sans la clé `service_role`). **OK** tant que la `SUPABASE_SERVICE_ROLE_KEY` est protégée.

4.7 Webhooks tiers

● `stripe-webhook`

Sécurité = signature HMAC Stripe vérifiée avec `STRIPE_WEBHOOK_SECRET`. **À confirmer** que la vérification est faite avec `stripe.webhooks.constructEvent(body, signature, secret)` et pas un simple compare.

5. Secrets management

5.1 Inventaire des secrets

Côté Supabase (secrets edge functions)

Secret	Usage	Si fuite
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Bypass RLS dans edge functions	🔴 CATASTROPHE : accès admin DB complet, lecture/écriture toutes tables tous tenants
<code>SUPABASE_JWT_SECRET</code> (côté Supabase)	Signature des JWTs	🔴 Forge n'importe quel JWT → impersonate n'importe qui
<code>RESEND_API_KEY</code>	Envoi emails	🟡 Spam au nom de LYTA / advisy. Pas d'accès data.
<code>TWILIO_AUTH_TOKEN</code>	SMS	🟡 SMS frauduleux + factures Twilio. Pas d'accès data.
<code>STRIPE_SECRET_KEY</code>	Stripe (mode live ?)	🔴 Refunds frauduleux, vol données customers Stripe
<code>STRIPE_WEBHOOK_SECRET</code>	Vérif signature webhooks	🟡 Forge faux webhooks → fausses updates abonnements
<code>OPENAI_API_KEY</code> / <code>ANTHROPIC_API_KEY</code>	Smartflow IA	🟡 Coûts IA explosent. Pas d'accès data direct.
<code>CLOUDFLARE_API_TOKEN</code>	DNS sous-domaines tenants	🔴 Take-over DNS de <code>*.lyta.ch</code>
<code>VERCEL_TOKEN</code>	Provisioning domaines Vercel	🟡 Manip déploiements
<code>ALLOWED_ORIGINS</code>	CORS whitelist	🟢 Pas un secret au sens strict

Côté Vercel (env vars frontend)

Variable	Usage	Risque si fuite
<code>VITE_SUPABASE_URL</code>	URL Supabase	🟢 Public de toute façon
<code>VITE_SUPABASE_ANON_KEY</code>	Clé anon Supabase	🟢 Public — déjà dans le bundle JS

Locaux (poste de Habib)

- `.env.local` (Vite) — vérifier qu'il est dans `.gitignore`
- `~/config/supabase/config.json` — auth CLI Supabase
- `.env.deploy` (Boreal) — credentials FTPS

5.2 Risques d'exposition actuels

🔴 Inspection vs `git log`

À auditer : aucun secret n'a-t-il été committé puis supprimé dans l'historique git ? Commande de check :

```
git log -p --all | grep -i -E "(api_key|service_role|secret|password|token)" | head -50
```

Si oui → rotation immédiate du secret en question (l'historique git public expose).

🟡 Logs edge functions

`createLogger` peut accidentellement logger un secret dans un message d'erreur (ex: stack trace). **À auditer** : `Deno.env.get` sont-elles jamais incluses dans les logs ?

🟡 Rotation

Pas de procédure documentée de rotation. À documenter : qui, quand, comment, qui notifier.

5.3 Recommandations secrets

1. **Audit historique git** (commande ci-dessus).
2. **Rotation périodique** : tous les secrets une fois par an minimum, immédiatement après tout départ d'un dev avec accès.
3. **Document de rotation** : qui rotate quoi.
4. **Monitoring usage** : alertes si pic de consommation Stripe/Resend/Twilio (signal compromise).
5. **Stripe mode test** : confirmer que `STRIPE_SECRET_KEY` est en mode `live` en prod uniquement. Idéalement utiliser deux comptes Stripe différents.

6. Multi-tenant isolation analysis

6.1 Modèle théorique

Tous les tenants partagent la même DB Postgres. Isolation via : 1. **Sous-domaine** (UI) — non sécurisé 2. **JWT** — claim `auth.uid()` après login 3. **RLS** — vraie isolation

6.2 Surface de pivotement

Peut-on, en tant que collab tenant A, accéder à des données du tenant B ?

Vecteurs testés en théorie

Vecteur	Risque	Statut
Direct <code>supabase.from('clients').select()</code>	❌ Bloqué par RLS (tenant_id)	🟢 OK si RLS correctes
Direct via RPC custom	⚠️ Si RPC fait SECURITY DEFINER sans check → bypass	🟡 À auditer
Storage paths	⚠️ Si on devine un path d'un autre tenant + RLS storage faible	🟡 À auditer
Edge function <code>bypass-insert</code>	⚠️ Si check tenant ne marche pas	🟡 À auditer
Edge function <code>create-client</code>	⚠️ Idem	🟡 À auditer
Edge function avec <code>service_role</code> qui prend un <code>tenant_id</code> en input sans vérifier	🔴 Pivot direct possible	🔴 À auditer URGENT
<code>king-impersonate-tenant</code> avec un JWT KING volé	🔴 Take-over total	🔴

Audit recommandé

Tester chaque edge function : peut-on l'appeler avec un JWT du tenant A et un `tenant_id` du tenant B dans le payload et obtenir une action sur le tenant B ?

Liste prioritaire : 1. `add-user-seat` — input `tenant_id` ? 2. `cancel-tenant-subscription` — input `tenant_id` ? 3. `bypass-insert` — vérif tenant ? 4. `create-client` — vérif tenant ? 5. `save-policy` — vérif tenant ? 6. `dispatch-mandat-to-companies` — input ID accès ?

6.3 RPC functions audit

Toutes les fonctions Postgres avec `SECURITY DEFINER` doivent être listées + auditées. Commande pour les sortir :

```
SELECT n.nspname, p.proname, pg_get_function_arguments(p.oid)
FROM pg_proc p JOIN pg_namespace n ON p.pronamespace = n.oid
WHERE p.prosecdef = true AND n.nspname = 'public';
```

À auditer une par une : ont-elles un check tenant explicite ?

7. Vulnérabilités connues ou suspectes

7.1 🔴 Confirmées critiques

V1 — Bug RLS 42501 cause inconnue

Description : INSERT sur `clients`, `policies`, `family_members`, `documents` plante en 42501 en runtime PostgREST alors que les conditions WITH CHECK semblent satisfaites en test SQL.

Impact actuel : workaround via 3 edge functions `service_role`. Marche mais **on insère via `service_role` = on bypass complètement les RLS**. Si une edge function de bypass a un bug d'auth → INSERT possible sur n'importe quel tenant.

Recommandation : investigation root cause prioritaire. Hypothèses à explorer : `current_setting('request.jwt.claims', true)` qui retourne NULL dans le contexte PostgREST runtime mais pas en CLI ; ou un mismatch de `search_path`.

V2 — `frame-ancestors` CSP ignoré

Description : `frame-ancestors 'none'` est délivré via `<meta>` mais les browsers ignorent cette directive en meta.

Impact : Clickjacking possible — un attaquant peut embarquer `*.lyta.ch` dans un `<iframe>` sur un site malveillant et tromper l'utilisateur.

Recommandation : déplacer en header HTTP via `vercel.json` :

```
{
  "headers": [{ "source": "/*.*", "headers": [
    { "key": "X-Frame-Options", "value": "DENY" },
    { "key": "Content-Security-Policy", "value": "frame-ancestors 'none'" }
  ]}]
}
```

7.2 🟡 Suspectes (à investiguer)

V3 — `king-impersonate-tenant` sans 2FA fresh

Description : si Habib est compromis (vol cookie, vol token), un attaquant peut impersonate n'importe quel tenant sans vérification supplémentaire.

Recommandation : exiger un re-prompt TOTP < 5 min pour cette fonction + notification visible côté tenant impersonné.

V4 — Reset password redirect URL

Description : `additional_redirect_urls` = `["https://lyta-xi.vercel.app/*", "https://app.lyta.ch/*", "https://*.lyta.ch/*"]`. Le wildcard `*.lyta.ch/*` autorise n'importe quel sous-chemin.

Impact : si un attaquant peut créer un sous-domaine sous lyta.ch (via un bug dans `tenant-onboarding`) → reset password vers domaine attaquant.

Recommandation : auditer le flow tenant-onboarding pour confirmer qu'un slug ne peut pas être pris arbitrairement.

V5 — Storage path traversal

Description : convention path `<broker_user_id>/<file>` dans bucket `documents`. Si les RLS storage ne valident pas strictement le préfixe → list/download cross-tenant.

Recommandation : tester avec un curl exhaustif (voir §3.3).

V6 — Logs edge functions exposent stack traces

Description : `console.error("[fn] failed", error)` peut exposer secrets en stack trace si le secret est inclus dans un message d'erreur.

Recommandation : audit des `createLogger` calls.

V7 — Pas de retry policy bounces Resend

Description : si un email mandat dispatché à une compagnie bounce, c'est marqué `bounced` mais pas retry. Le broker peut ne pas s'en rendre compte → mandat jamais reçu par la compagnie → client se croit en règle.

Impact : **risque métier** (pas sécurité au sens strict).

Recommandation : alert visible dans CRM si bounce sur un dispatch.

V8 — `commission_statement_lines` non typé

Description : type-gen Supabase out-of-sync → cast `as any`. Si demain un dev change la struct sans s'en rendre compte côté front → bug silencieux possible.

Recommandation : régénérer types + ESLint rule `no-explicit-any`.

V9 — `clients_safe` view orpheline

Description : vue Postgres présente, usage actuel inconnu. Si elle masque des colonnes via une logique différente des RLS → divergence + risque que le front se rapaie sur elle et que la vue ait un bug.

Recommandation : confirmer usage et soit drop soit re-vérifier la logique.

7.3 🟢 Mineures / hygiène

- Pas de Sentry → debug compliqué en cas d'incident
 - Pas de tests automatisés → régression facile
 - Pas de Husky → pas de garde-fou avant push
 - Stripe price ID hardcodé en fallback dans `add-user-seat`
 - Doublons table `audit_log(s)` et `king_audit_log(s)` → confusion possible sur "quelle table contient quoi"
-

8. Recommandations prioritaires

8.1 P0 — À faire en priorité absolue (semaine 1)

#	Action	Effort
1	Auditer l'historique git pour exposition de secrets (commande §5.2)	30 min
2	Rotation <code>SUPABASE_SERVICE_ROLE_KEY</code> + <code>STRIPE_SECRET_KEY</code> après l'audit dev (ou immédiatement si fuite détectée)	1h
3	Investiguer le bug RLS 42501 (V1)	1-2 jours
4	Déplacer <code>frame-ancestors</code> en header HTTP Vercel (V2)	30 min
5	Auditer <code>king-impersonate-tenant</code> (V3) — ajouter 2FA fresh + notification	4h
6	Tester pivot multi-tenant sur les 6 fonctions prioritaires (§6.2)	1 jour

8.2 P1 — Sprint 1 (semaines 2-3)

#	Action	Effort
7	Audit ligne par ligne des 16 fonctions REQUIREAUTH (§4.1)	2 jours
8	Audit RPC SECURITY DEFINER (§6.3)	1 jour
9	Test path traversal Storage (V5)	4h
10	Cleanup tables doublons audit (§7.3)	1 jour
11	Régénérer types Supabase (V8)	30 min
12	Setup Sentry frontend + logs centralisés	4h

8.3 P2 — Sprint 2-3

#	Action
13	Setup Husky + ESLint pre-commit
14	Setup Vitest + Playwright + premier suite de tests sur flows critiques
15	Documenter procédure rotation des secrets
16	Alertes monitoring (Stripe/Resend/Twilio pic anormal)
17	Implémenter retry policy bounces Resend
18	Migration <code>clients_safe</code> (drop ou refactor)
19	Migration <code>contracts</code> legacy (audit usage + drop)
20	Découper <code>KingWizard.tsx</code> (1158 LOC) en sous-composants

8.4 Long terme

- **SOC 2 Type II** : si LYTA vise les clients corporate, audit SOC 2 nécessaire (~6-12 mois)
- **Pen test externe** : tous les 6 mois minimum
- **Bug bounty program** : intéressant à terme

9. Annexe — Procédure de réponse à incident sécurité

En cas de détection d’une compromission (vol JWT, accès non autorisé détecté, secret leak) :

- 1. Containment :**
 - Rotation immédiate du secret compromis
 - Force logout tous les users via Supabase Auth → “Revoke all sessions”
 - Bloquer le tenant concerné si confirmé
- 2. Investigation :**
 - Logs Supabase + Vercel sur la fenêtre suspecte
 - `audit_logs` + `king_audit_logs`
- 3. Notification :**
 - Habib + dev concerné
 - Tenants impactés (obligation nLPD : 72h)
 - Préposé fédéral à la protection des données (PFPDT) si data critique
- 4. Postmortem :**
 - Root cause analysis

- Plan d'action correctif
 - Documenter dans `Documentation/incidents/` (à créer)
-

Document confidentiel. Diffusion restreinte. Toute copie doit être tracée. — Optimislink Sàrl, 8 juin 2026.